

Linux Permission Audit Documentation

1. Introduction

Linux permissions are a core security mechanism used to control access to files and directories. They define who can view, modify, or execute system resources. In both on-premise and cloud-based Linux environments, permissions play a critical role in system security, auditing, and access control.

This documentation explains Linux permissions in a structured and practical way. It covers how permissions work, how to identify them, how users and groups interact with files, and how permissions are assigned using both symbolic and numeric methods. The focus is on understanding, not memorization.

2. Linux Permission Basics

Linux permissions are built around three actions:

- Read (r): Read permission allows a user to view the contents of a file. For directories, it allows listing the directory contents.
- Write (w): Write permission allows modifying a file. In directories, it allows creating, deleting, or renaming files within that directory.
- Execute (x): Execute permission allows running a file as a program. For directories, it allows entering the directory and accessing its contents.

These permissions apply to every file and directory in a Linux system.

3. File Types in Linux

Linux does not rely on file extensions to determine file types. Instead, it uses metadata.

There are two main types relevant to permissions:

1. Directories: Directories are containers that hold files and other directories. They are similar to folders but require execute permission to be accessed
2. Non-directories (regular files): These include text files, scripts, and binaries.

A file's type is identified using the (**ls -l**) command. The first character of the permission string indicates the file type:

- **d** means directory
- **-** means regular file

Note: A .txt extension does not mean something is not a directory, and the absence of an extension does not mean something is executable.

4. Permission Structure (Four Octets)

When permissions are displayed using **ls -l**, they appear in a structured format, for example:

(**drwxr-xr-**)

This structure can be divided into four parts:

1. File type indicator: This is the first character (d or -)
2. User (owner) permissions: The next three characters define what the owner of the file can do.
3. Group permissions: The following three characters define what members of the file's group can do.
4. Other permissions: The last three characters define what everyone else can do.

Each permission set contains read, write, and execute in that order.

5. Users, Groups, and Others

Linux permissions are always evaluated in this order:

1. User (Owner): The user who owns the file. This is usually the account that created it.
2. Group: A collection of users. When a user is created, Linux typically creates a group with the same name.
3. Others: Any user who is neither the owner nor a member of the group.

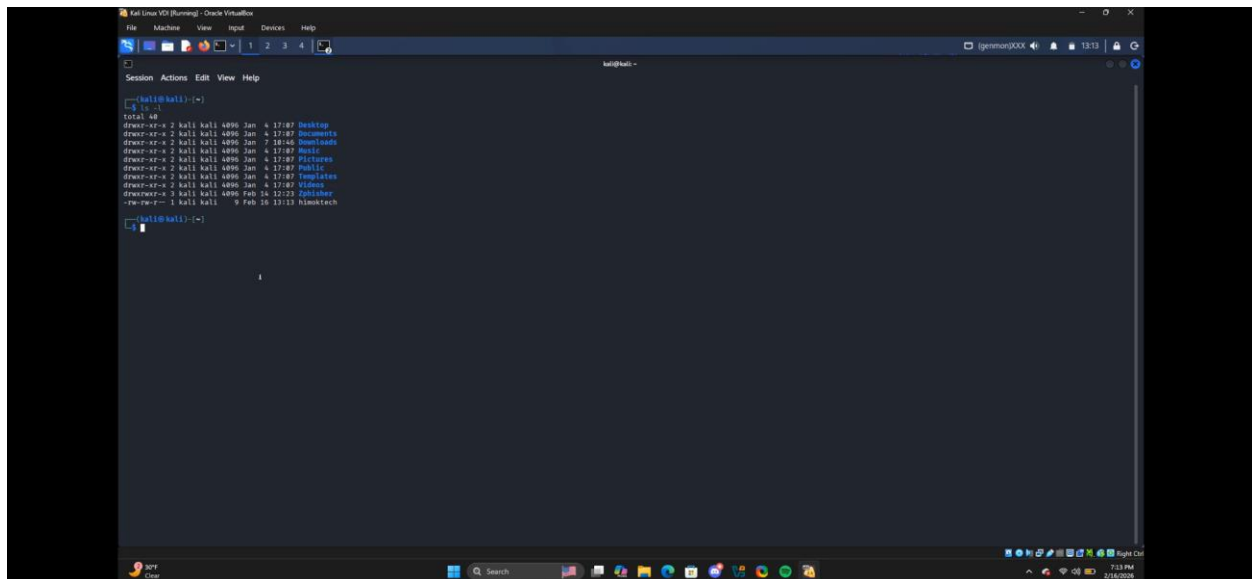
This structure allows efficient access control. Instead of assigning permissions to individual users, administrators can assign permissions to groups, automatically affecting all group members.

6. Identifying Permissions

To view permissions, ownership, and file types, the following command is used:

ls -l : This command is essential during permission audits. It shows:

- File type
- User owner
- Group owner
- Permission sets
- File names



```
kali@kali:~$ ls -l
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 2 kali kali 4096 Feb 14 12:23 ZipArchive
-rw-rw-r-- 1 kali kali  9 Feb 16 13:13 .viminfo
```

Color coding in the terminal is not a reliable method for identifying file types. The permission string is the standard and correct approach.

7. Why Permissions Matter in Security Audits

From a security perspective, Linux permissions are used to:

- Restrict access to sensitive data
- Prevent unauthorized modification of system files
- Limit execution of scripts and binaries
- Enforce the principle of least privilege
- Reduce attack surfaces in cloud and enterprise environments

Poorly configured permissions are a common cause of privilege escalation and system compromise.

8. Methods of Assigning Permissions

Linux provides two methods for assigning permissions using the chmod command:

1. Symbolic method
2. Numeric (octal) method

Both methods modify the same permission system but use different representations.

9. Symbolic Method

The symbolic method uses letters and operators.

User classes:

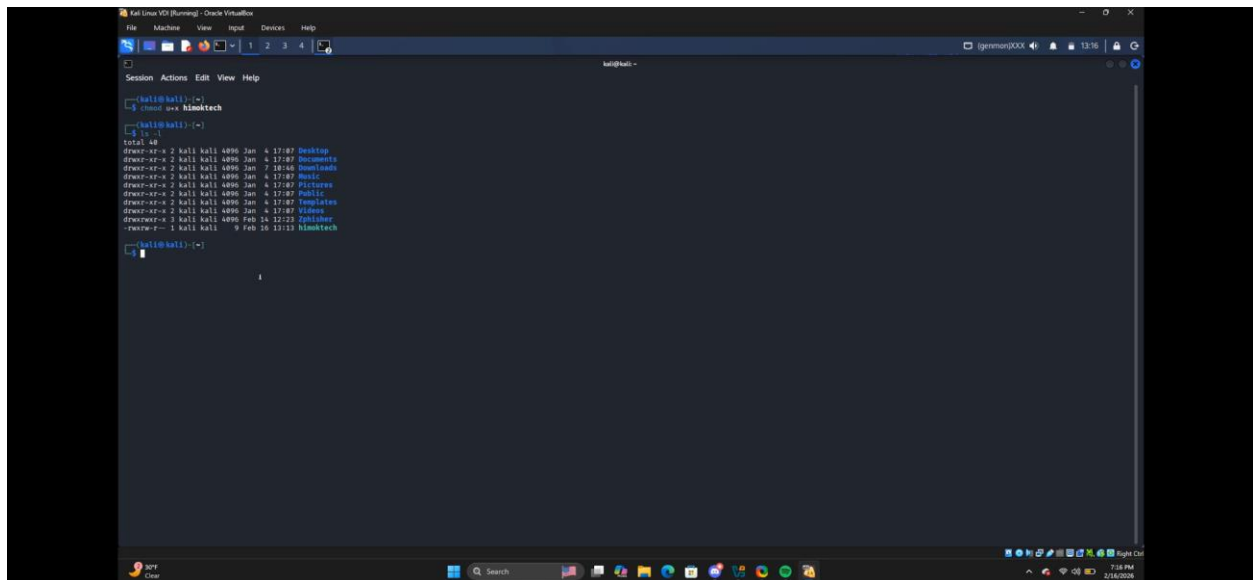
- u for user (owner)
- g for group
- o for others
- a for all

Operators:

- + to add permissions
- - to remove permissions
- = to assign exact permissions

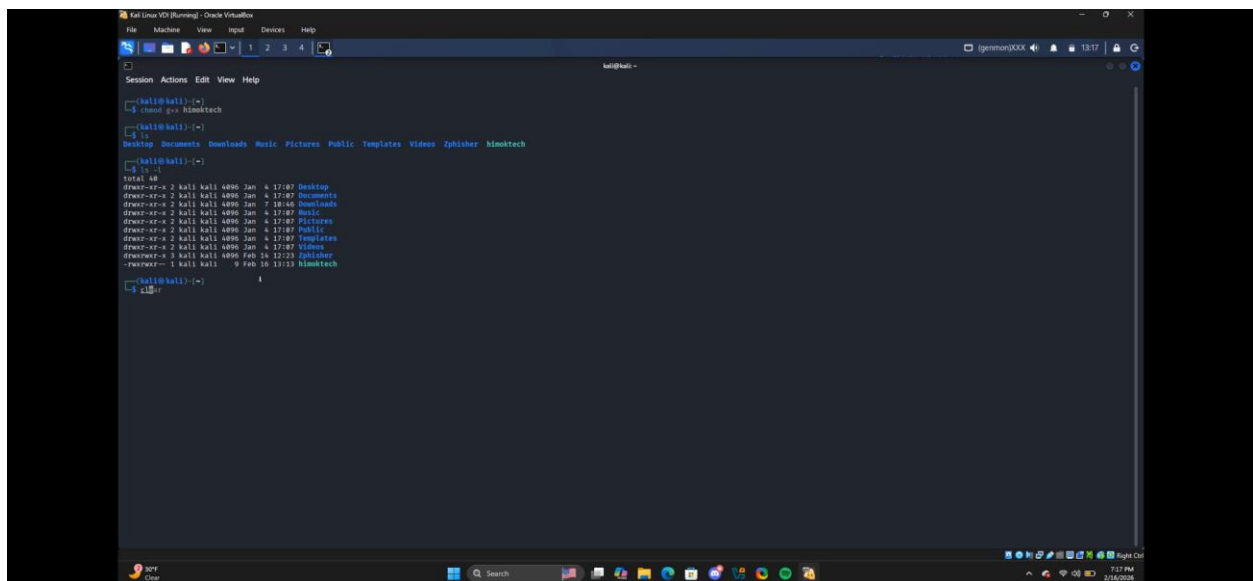
Examples:

Add execute permission to the user: `chmod u+x filename`



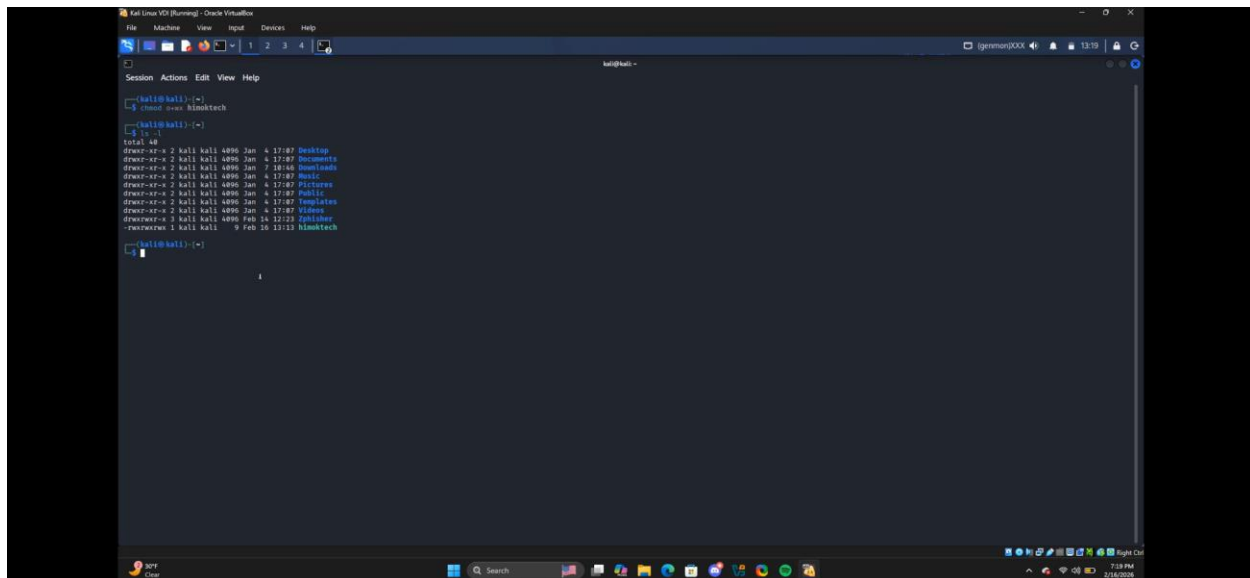
```
kali@kali:~$ chmod u+x h1mktech
kali@kali:~$ ls -la
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  7 10:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxrwxr-x 3 kali kali 4096 Feb 26 12:22 ZshHistory
-rwxrwxr-x 1 kali kali  0 Feb 26 12:13 h1mktech
kali@kali:~$
```

Add execute permission to the group: `chmod g+x filename`



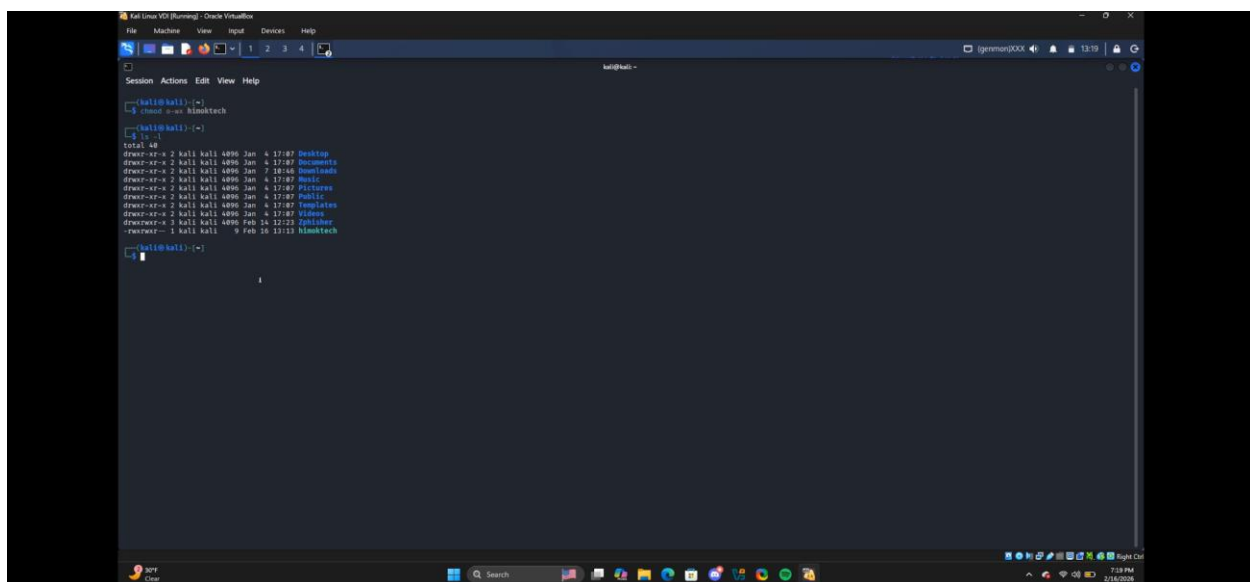
```
kali@kali:~$ chmod g+x h1mktech
kali@kali:~$ ls -la
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  7 10:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxrwxr-x 3 kali kali 4096 Feb 26 12:22 ZshHistory
-rwxrwxr-x 1 kali kali  0 Feb 26 12:13 h1mktech
kali@kali:~$
```

Add write and execute permission to others: `chmod o+wx filename`



```
kali@kali:~$ ls -la
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  2 10:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 3 kali kali 4096 Feb 14 12:23 ZzzZzzz
-rwxrwxr-x 1 kali kali  9 Feb 16 13:13 himektech
```

Remove write and execute permission from others: `chmod o-wx filename`



```
kali@kali:~$ ls -la
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  2 10:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 3 kali kali 4096 Feb 14 12:23 ZzzZzzz
-rwxr-xr-x 1 kali kali  9 Feb 16 13:13 himektech
```

Remove read permission from all: `chmod a-r filename`

```
kali@kali:~$ sudo chmod a=r /home/hacker
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  2 16:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 3 kali kali 4096 Feb 14 12:23 Zzzdow
-rw-r--r-- 1 kali kali  9 Feb 16 13:13 himektech
```

Assign read-only permission to all users: `chmod a=r filename`

```
kali@kali:~$ sudo chmod a=r /home/hacker
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  2 16:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 3 kali kali 4096 Feb 14 12:23 Zzzdow
-rw-r--r-- 1 kali kali  9 Feb 16 13:13 himektech
```

The symbolic method is readable and useful when making precise, incremental permission changes.

10. Numeric (Octal) Method

The numeric method assigns permissions using numbers based on binary values:

Read = 4

Write = 2

Execute = 1

Permissions are calculated by adding these values.

Common permission values:

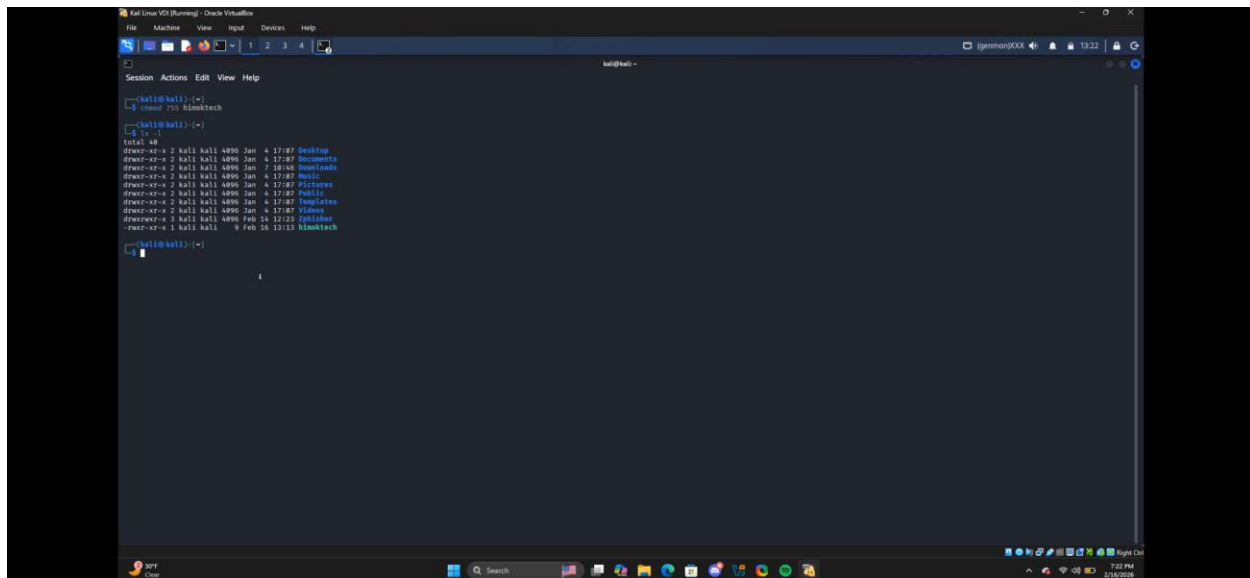
- 7 = read, write, execute
- 6 = read, write
- 5 = read, execute
- 4 = read only
- 3 = write, execute
- 1 = execute only
- 0 = no permissions

Permissions are written as three digits: User | Group | Others

Example: **chmod 755 filename**

This gives:

- User: read, write, execute
- Group: read, execute
- Others: read, execute

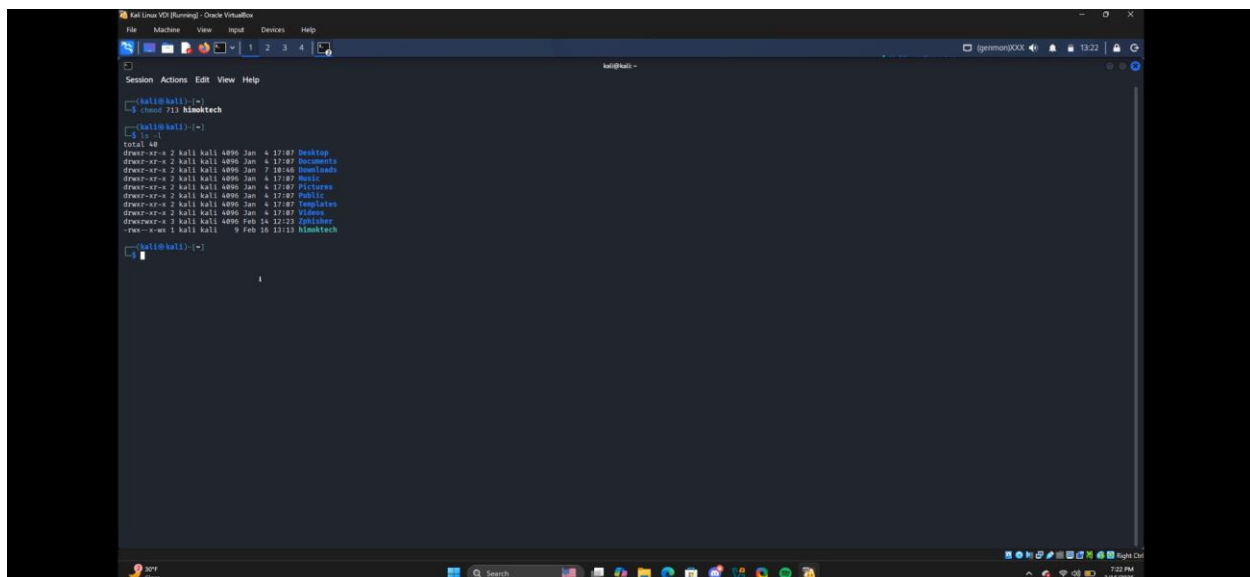


```
Kali Linux VM (Running) - Oracle VM VirtualBox
File Machine View Input Devices Help
kali@kali ~
Session Actions Edit View Help
kali@kali:~$ chmod 755 himsketch
kali@kali:~$ ls -l
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  7 18:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 3 kali kali 4096 Feb 14 12:23 Zzzisher
-rwxr-xr-x 1 kali kali  9 Feb 16 13:13 himsketch
kali@kali:~$
```

Another example: **chmod 713 filename**

This gives:

- User: read, write, execute
- Group: execute only
- Others: write and execute



```
Kali Linux VM (Running) - Oracle VM VirtualBox
File Machine View Input Devices Help
kali@kali ~
Session Actions Edit View Help
kali@kali:~$ chmod 713 himsketch
kali@kali:~$ ls -l
total 48
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Desktop
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Documents
drwxr-xr-x 2 kali kali 4096 Jan  7 18:16 Downloads
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Music
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Pictures
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Public
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Templates
drwxr-xr-x 2 kali kali 4096 Jan  4 17:07 Videos
drwxr-xr-x 3 kali kali 4096 Feb 14 12:23 Zzzisher
-rwx--x-w- 1 kali kali  9 Feb 16 13:13 himsketch
kali@kali:~$
```

The numeric method is faster and commonly used in scripting and automation.

11. Verification After Changes

After modifying permissions, verification is mandatory.

This is done using: **ls -l**

Verification ensures:

- Permissions were applied correctly
- No unintended access was introduced
- System functionality remains intact

Skipping verification is a common auditing mistake.

12. Cloud and IAM Perspective

Linux permissions map directly to cloud security concepts.

- Linux users resemble IAM identities
- Linux groups resemble IAM groups or roles
- File permissions resemble IAM policies
- chmod actions resemble policy changes

Understanding Linux permissions makes cloud IAM concepts easier to understand and apply correctly.

13. Offensive Security and Misconfiguration Awareness

Common dangerous configurations include:

- World-writable files (777)
- Writable executable scripts
- Weak group permissions
- Misconfigured directories such as /tmp

From an attacker's perspective, misconfigured permissions enable privilege escalation and lateral movement. From a defender's perspective, regular permission audits prevent these issues before exploitation occurs.

14. Linux Permission Audit Checklist

During an audit:

- Identify file owners and confirm correctness
- Review group membership and purpose
- Detect excessive permissions
- Enforce least privilege
- Remove unnecessary execute permissions
- Verify changes and document results

15. Conclusion

Linux permissions are simple in design but powerful in impact. Proper understanding and careful management of permissions are essential for system security, auditing, and cloud operations.

Both symbolic and numeric methods are valid. The key is knowing when and how to use them correctly. Regular audits, verification, and least-privilege enforcement remain the most effective practices.